

Manual de Usuario

Sistema de Recomendación de Restaurantes con Neo4j

Información general

- **Estudiante:** Ludwing Alexander López Ortiz
 - **Carné:** 201907608
 - **Grupo:** G18
 - **Curso:** Sistemas de Bases de Datos 2
-

1. Objetivo del manual

Este manual explica cómo instalar, configurar y ejecutar el sistema de recomendación de restaurantes.

También describe cómo cargar los datos, ejecutar las consultas Cypher, visualizar el grafo e interpretar los resultados principales.

2. Requisitos

Se recomienda utilizar Linux y contar con las siguientes herramientas:

- Docker.
- Docker Compose.
- Python 3.8 o superior.
- pip.
- Git.
- Navegador web.
- Visual Studio Code o un editor equivalente.

Verificación:

```
docker --version
docker compose version
python3 --version
pip3 --version
git --version
```

3. Obtener el proyecto

Clonar el repositorio:

```
git clone https://github.com/Ahletz/BD2_VACJUN26_Proyectos_G18.git
```

Ingresar al proyecto:

```
cd "BD2_VACJUN26_Proyectos_G18/Segundo Proyecto"
```

4. Configurar las variables de entorno

Crear el archivo `.env`:

```
cp .env.example .env
```

Editar el archivo:

```
nano .env
```

Ejemplo:

```
NEO4J_URI=neo4j://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=CONTRASENA_SEGURA
NEO4J_DATABASE=neo4j
```

El archivo `.env` no debe subirse al repositorio.

5. Iniciar Neo4j

Ejecutar:

```
docker compose up -d
```

Verificar el contenedor:

```
docker ps
```

Revisar los registros:

```
docker compose logs neo4j
```

El servidor estará disponible cuando aparezcan mensajes similares a:

```
Bolt enabled on 0.0.0.0:7687
HTTP enabled on 0.0.0.0:7474
Started
```

6. Ingresar a Neo4j Browser

Abrir:

```
http://localhost:7474
```

Datos de conexión:

```
Connection URL: neo4j://localhost:7687
Username: neo4j
Password: contraseña definida en .env
```

Después de conectarse, las consultas pueden escribirse en la barra superior de Neo4j Browser.

Para ejecutar una consulta se debe presionar el botón de reproducción o utilizar:

```
Ctrl + Enter
```

7. Preparar Python

Crear el entorno virtual:

```
python3 -m venv venv
```

Activarlo:

```
source venv/bin/activate
```

Instalar las dependencias:

```
python -m pip install -r requirements.txt
```

Probar la conexión:

```
python scripts/python/probar_conexion.py
```

Resultado esperado:

```
Conexión exitosa con Neo4j
```

8. Generar los datos

Ejecutar:

```
python scripts/python/generar_datos.py
```

Se generarán los archivos CSV en:

```
datos/nodos/  
datos/relaciones/
```

Cantidades mínimas:

- 500 usuarios.
- 200 restaurantes.
- 100 chefs.
- 50 platillos.
- 15 tipos de cocina.

9. Copiar los CSV al directorio de importación

Cuando se utiliza el contenedor creado manualmente, copiar los archivos:

```
mkdir -p "$HOME/neo4j-restaurantes/import/nodos"  
mkdir -p "$HOME/neo4j-restaurantes/import/relaciones"  
  
cp datos/nodos/*.csv "$HOME/neo4j-restaurantes/import/nodos/"  
cp datos/relaciones/*.csv "$HOME/neo4j-restaurantes/import/relaciones/"
```

Cuando se utiliza Docker Compose, la carpeta **datos** se monta automáticamente dentro del contenedor.

10. Crear restricciones e índices

Ejecutar:

```
docker exec -i neo4j-restaurantes \  
  cypher-shell \  
  -u neo4j \  
  -p CONTRASENA \  
  -d neo4j \  
  < scripts/cypher/01_esquema.cypher
```

Verificar restricciones:

```
SHOW CONSTRAINTS;
```

Verificar índices:

```
SHOW INDEXES;
```

11. Cargar nodos

Ejecutar:

```
docker exec -i neo4j-restaurantes \  
  cypher-shell \  
  -u neo4j \  
  -p CONTRASENA \  
  -d neo4j \  
  < scripts/cypher/02_carga_nodos.cypher
```

Validar:

```
MATCH (n)  
RETURN labels(n)[0] AS tipo, count(n) AS cantidad  
ORDER BY tipo;
```

Resultado esperado:

Tipo	Cantidad
Chef	100
Platillo	50
Restaurante	200
TipoCocina	15
Usuario	500

12. Cargar relaciones

Ejecutar:

```
docker exec -i neo4j-restaurantes \
  cypher-shell \
  -u neo4j \
  -p CONTRASENA \
  -d neo4j \
  < scripts/cypher/03_carga_relaciones.cypher
```

Validar:

```
MATCH ()-[r]->()
RETURN type(r) AS relacion, count(r) AS cantidad
ORDER BY relacion;
```

13. Ejecutar validaciones

```
docker exec -i neo4j-restaurantes \
  cypher-shell \
  -u neo4j \
  -p CONTRASENA \
  -d neo4j \
  < scripts/cypher/04_validaciones.cypher
```

Los siguientes resultados deben ser cero:

- Nodos sin identificador.
- Calificaciones fuera del rango.
- Ofertas duplicadas.
- Visitas con consumo inválido.
- Restaurantes sin tipo de cocina.

- Restaurantes sin platillos.

14. Ejecutar las consultas obligatorias

Desde la terminal:

```
docker exec -i neo4j-restaurantes \
  cypher-shell \
  -u neo4j \
  -p CONTRASENA \
  -d neo4j \
  < consultas/01_consultas_obligatorias.cypher
```

También pueden copiarse individualmente en Neo4j Browser.

15. Interpretación de las consultas

Diversidad de cocina

Muestra los restaurantes con mayor cantidad de tipos de cocina.

Un valor más alto indica una oferta gastronómica más variada.

Tasa de reservas

Muestra el porcentaje de visitas realizadas con reserva.

Ejemplo:

```
88.24
```

Significa que el 88.24 % de las visitas se realizaron con reserva.

Gasto promedio

Muestra cuánto consume en promedio cada usuario durante una visita.

Frecuencia de visitas

Muestra qué usuarios visitaron más restaurantes durante los últimos 180 días.

Restaurantes sin visitas recientes

Permite detectar establecimientos que no han recibido visitas durante un periodo determinado.

Movilidad de chefs

Muestra la cantidad de restaurantes en los que trabajó cada chef.

Variación de precios

Muestra la diferencia entre el precio mínimo y máximo de un mismo platillo.

Crecimiento por tipo de cocina

Compara la cantidad de visitas de dos periodos.

Un resultado positivo representa crecimiento y uno negativo representa reducción.

Recomendación

Muestra restaurantes que comparten chefs con establecimientos bien valorados por el usuario.

No se recomiendan restaurantes que el usuario ya visitó.

16. Ejecutar análisis de redes

```
docker exec -i neo4j-restaurantes \
  cypher-shell \
  -u neo4j \
  -p CONTRASENA \
  -d neo4j \
  < consultas/02_analisis_redes.cypher
```

Ruta más corta

La propiedad `grados_separacion` indica la cantidad de relaciones de amistad entre dos usuarios.

Restaurantes conectados

El valor `nivel_conectividad` combina chefs, platillos disponibles y calificaciones.

17. Visualizar el grafo

Abrir Neo4j Browser y ejecutar:

```
MATCH ruta = shortestPath(
  (:Usuario {id: 1})-[:ES_AMIGO_DE*..10]-(:Usuario {id: 2})
)
RETURN ruta;
```

Seleccionar la vista:

Graph

Los nodos pueden moverse arrastrándolos con el cursor.

18. Consulta de recomendación

```
MATCH ruta = (
  usuario:Usuario {id: 1}
)-[calificacion:`CALIFICÓ`]->(
  restaurante_valorado:Restaurante
)<-[:TRABAJA_EN]-(
  chef:Chef
)-[:TRABAJA_EN]->(
  restaurante_recomendado:Restaurante
)
WHERE calificacion.puntuacion >= 4
AND restaurante_recomendado <> restaurante_valorado
AND NOT EXISTS {
  MATCH (usuario)-[:`VISITÓ`]->(restaurante_recomendado)
}
RETURN ruta
LIMIT 5;
```

El resultado muestra:

- Usuario evaluado.
- Restaurante bien calificado.
- Chef compartido.
- Restaurante recomendado.

19. Comandos de administración

Iniciar:

```
docker compose up -d
```

Detener:

```
docker compose stop
```

Reiniciar:

```
docker compose restart
```

Ver registros:

```
docker compose logs -f neo4j
```

Apagar y eliminar el contenedor:

```
docker compose down
```

Eliminar también los datos almacenados:

```
docker compose down -v
```

El último comando elimina permanentemente la base de datos.

20. Problemas comunes

Neo4j Browser no abre

Verificar el contenedor:

```
docker ps
```

Revisar los registros:

```
docker logs neo4j-restaurantes
```

Error de autenticación

Verificar:

- Usuario.
- Contraseña.
- Archivo `.env`.
- Variable `NEO4J_AUTH`.

Los CSV no son encontrados

Verificar los archivos:

```
docker exec neo4j-restaurantes find /import -type f
```

El puerto ya está ocupado

Verificar:

```
sudo lsof -i :7474  
sudo lsof -i :7687
```

El contenedor ya existe

Eliminar el contenedor anterior:

```
docker rm -f neo4j-restaurantes
```

Luego ejecutar nuevamente:

```
docker compose up -d
```

El entorno virtual no está activo

Activar:

```
source venv/bin/activate
```

21. Evidencias

Los resultados se encuentran en:

```
resultados/
```

Las capturas se encuentran en:

```
resultados/capturas/
```

El modelo visual se encuentra en:

```
diagramas/modelo_grafo_restaurantes.png
```

22. Cierre del sistema

Al finalizar el trabajo:

```
docker compose stop
```

Para volver a iniciar:

```
docker compose start
```